

Introduction to snnlang

ar063 · 30 July 2026 · Draft

What snnlang does

snnlang is the network-authoring layer for *tools/snn*. It provides a small Python API for describing populations, projections, inputs, outputs, and recordings. Compilation checks the description and writes a portable, data-only bundle. *tools/snn* loads that bundle and performs the simulation or training.

The system separates four concerns:

1. The **graph** describes reusable computation.
2. The optional **training recipe** describes standard learning choices.
3. The **execution protocol** supplies data, duration, seeds, device, checkpoints, and recordings for one run.
4. The **experiment** owns the scientific question, conditions, analysis, figures, and conclusions.

A saved bundle contains scientific structure rather than live Python objects. It can be inspected and replayed without importing the authoring package.

Your first network

A network begins with an explicit timestep and a typed input. Components add reusable motifs. Outputs are values returned to a caller; observables are internal signals retained for inspection.

```
from tools import snnlang as snn

net = snn.Network("small_ping", dt=0.1 * snn.ms)
events = net.input(
    "events",
    shape=("time", "batch", 128),
    signal_type="spikes",
    unit="spike",
)

cell = snn.components.ping(
    net,
    name="cell",
    n_e=80,
    n_i=20,
    source=events,
)
net.expose(cell.E.spikes, cell.I.spikes, name="raster")
```

```
bundle = snn.compile(net, target="tools/snn")
bundle.write("small_ping.bundle", visualise=True)
```

Compilation produces `graph.json`, `manifest.json`, a readable report, and optional circuit diagrams.

Simulate it

Provide one tensor for every declared input:

```
import torch
from tools.snn.execution import ExecutionSpec, simulate

spikes = torch.zeros(2_000, 1, 128)
result = simulate(ExecutionSpec(
    kind="simulate",
    executor="graph",
    bundle="small_ping.bundle",
    inputs={"events": spikes},
    seed=42,
    device="cpu",
    recording="observables",
))

e_spikes = result.recordings["raster_0"]
i_spikes = result.recordings["raster_1"]
```

Graph-native forward simulation supports this example today. Input generation remains the caller's responsibility.

Continue reading

The remaining pages in the SNNLANG collection cover authoring, graph composition, compilation and execution, readouts and inputs, training, state and provenance, and safe extension. Each page states what is implemented and what remains planned.

Next: Networks, signals, and parameters